Lab 3 Report

Name. MAlik Musawar Ali Roll num .su92-bssem-s25-006 section. BSSEM 4A

1. Problem and Assumptions

In this lab I traced BFS and DFS by hand on a graph with nodes A to G, implemented DLS and IDDFS, and then used UCS on a weighted version of the graph. The start is A and the goal is G.

The graph is undirected and has these edges. A-B, A-C, B-D, B-E, C-F, E-G and F-G. The successor order is the order of the neighbours. A has B, C. B has A, D, E. C has A, F. D has B. E has B, G. F has C, G. G has E, F. The root is at depth 0.

For BFS a node is marked visited when it is added to the queue. For DFS the stack is used, successors are pushed in reverse so the first successor is processed first, and a node is marked when it is processed. Both stop when the goal is processed. The code is my own and no outside code was copied.

2. Manual BFS Trace

- Step 1. Process A, path A. Queue after is B, C.
- Step 2. Process B, path A-B. Queue after is C, D, E.
- Step 3. Process C, path A-C. Queue after is D, E, F.
- Step 4. Process D, path A-B-D. Queue after is E, F.
- Step 5. Process E, path A-B-E. Queue after is F, G.
- Step 6. Process F, path A-C-F. G is already in the queue. Queue after is G.
- Step 7. Process G, path A-B-E-G. Goal reached, stop.

Final path is A-B-E-G. Visit order is A, B, C, D, E, F, G.

3. Manual DFS Trace

- Step 1. Process A, path A. Stack after is C, B (B is the top).
- Step 2. Process B, path A-B. Stack after is C, E, D (D is the top).
- Step 3. Process D, path A-B-D. D has no new neighbour. Stack after is C, E.
- Step 4. Process E, path A-B-E. Stack after is C, G.
- Step 5. Process G, path A-B-E-G. Goal reached, stop.

Final path is A-B-E-G. Visit order is A, B, D, E, G.

I ran lab3_bfs_dfs.py and the printed steps, paths, final paths and visit orders are the same as my manual traces. Both give the same path, but DFS processed 5 nodes and BFS processed 7.

4. DLS and IDDFS

DLS checks cycles only on the current path. The status is FOUND, CUTOFF or FAILURE. Results from the code.

- Limit 1. CUTOFF, no path. Visit order A, B, C.
- Limit 2. CUTOFF, no path. Visit order A, B, D, E, C, F.
- Limit 3. FOUND, path A-B-E-G. Visit order A, B, D, E, G.
- Limit 4. FOUND, path A-B-E-G. Visit order A, B, D, E, G.

IDDFS tries limits from 0 and stops when it finds the goal.

- Max depth 1. Limits attempted 0, 1. CUTOFF, no path. Total nodes visited 4.
- Max depth 2. Limits attempted 0, 1, 2. CUTOFF, no path. Total nodes visited 10.
- Max depth 3. Limits attempted 0, 1, 2, 3. FOUND, path A-B-E-G. Total nodes visited 15.
- Max depth 4. Limits attempted 0, 1, 2, 3 and it stops at 3. FOUND, path A-B-E-G. Total nodes visited 15.

Repeated work. IDDFS starts again from A for every limit, so A is visited in every attempt and B is visited again many times. The last attempt alone visits 5 nodes, but IDDFS visited 15 in total. This extra work is the price for using little memory.

Why limits below 3 do not succeed. G is at depth 3, because the shortest path A-B-E-G has 3 edges. With limit 1 or 2 the search is stopped before it reaches depth 3, so it gives CUTOFF.

5. Weighted Graph and UCS

I made a weighted version of the same graph. The weights are my own assumptions for this lab. I also added one corridor, D to E with weight 1, because the original graph does not have it. Edge weights.

- A to B weight 2.
- A to C weight 1.
- B to D weight 1.
- B to E weight 6.
- C to F weight 2.
- D to E weight 1 (added corridor).
- E to G weight 5.
- F to G weight 7.

UCS processing order from the code is A (cost 0), C (1), B (2), F (3), D (3), E (4) and G (9). The older entry for E with cost 8 came out later and was skipped because E was already processed. UCS returned the path A-B-D-E-G with cost 9.

Independent check. I added the weights again in a separate function. A-B is 2, B-D is 1, D-E is 1 and E-G is 5, so the total is 9, the same as UCS.

BFS on the same weighted graph gives A-B-E-G with 3 hops. Its cost is 2 + 6 + 5 = 13. The UCS path has 4 hops but costs only 9. A route with fewer hops can cost more because BFS counts the number of edges and ignores the weights. The edge B to E is expensive, and the longer way through D uses cheap edges.

6. Comparison of the Alternatives

BFS and DFS found the same path here, but DFS visited fewer nodes. This is not always true, because DFS can also go deep in a wrong branch and can find a longer path. DLS is DFS with a limit, so it uses little memory but gives CUTOFF if the limit is small. IDDFS repeats the work but it finds the shallowest goal like BFS. BFS finds the fewest hops and UCS finds the lowest cost, and they are the same only when all the edge weights are equal.

7. Failure / Edge Case

Input. DLS from A to G with limit 2, and IDDFS with max depth 2.

Observed behaviour. The status is CUTOFF and no path is returned.

Likely cause. G is at depth 3 which is below the limit, so the search was stopped by the limit. It did not check the whole graph. CUTOFF is different from FAILURE, and CUTOFF does not mean that no path exists.

Proposed improvement. Increase the limit step by step with IDDFS, or use a bigger max depth, and only say that there is no path when the status is FAILURE.

A second small edge case is in UCS. The same node E was in the priority queue two times with costs 4 and 8. The cost 4 came first and the second one was skipped, so the answer is not wrong.

8. Limitations

I used only one small graph with 7 nodes and one added corridor. The weights are made up, so the answers can change with other weights. I counted node visits, not real running time. My results depend on the successor order and on the way I mark visited nodes.

9. Reflection and Responsible Practice

[Write your reflection here in your own words. For example, what was difficult in the trace and what you learned about BFS and UCS.]

- All data is made up. No real private information or passwords are uploaded to GitHub.
- The assumptions about the graph, the weights and the added corridor are written clearly, so they are not mistaken for real data.
- No outside code was copied. The search ideas are from the reference below.